

UNIVERSIDADE ESTADUAL DO NORTE FLUMINENSE
LABORATÓRIO DE ENGENHARIA E EXPLORAÇÃO DE PETRÓLEO
CENTRO DE CIÊNCIA E TECNOLOGIA

PROJETO DE ENGENHARIA
DESENVOLVIMENTO DO SOFTWARE
PoreElasticCalculator

DISCIPLINA: Projeto de Software Aplicado à Engenharia

Matheus Bastos & Nicolau Prates
Prof. André Duarte Bueno

MACAÉ - RJ
Junho - 2025

Sumário

1	Concepção	6
1.1	Metodologia	6
1.2	Nome do Sistema/Produto	7
1.3	Especificação	7
1.4	Requisitos	7
1.4.1	Requisitos funcionais	7
1.4.2	Requisitos não funcionais	7
1.5	Casos de Uso	8
1.5.1	Diagrama de caso de uso geral	8
1.5.2	Diagrama de caso de uso específico	8
2	Elaboração	10
2.1	Análise de domínio	10
2.2	Formulação teórica	10
2.2.1	Modelo de Voigt-Reuss-Hill (VRH)	10
2.2.2	Limites de Hashin-Shtrikman (HS)	11
2.3	Identificação de pacotes – assuntos	11
2.4	Diagrama de pacotes – assuntos	12
3	AOO – Análise Orientada a Objeto	13
3.1	Diagramas de classes	13
3.1.1	Dicionário de classes	13
3.2	Diagrama de sequência – eventos e mensagens	15
3.2.1	Diagrama de sequência geral	15
3.2.2	Diagrama de sequência específico	16
3.3	Diagrama de comunicação – colaboração	17
3.4	Diagrama de estado	17
3.5	Diagrama de atividades	18
4	Projeto	20
4.1	Projeto do Sistema	20
4.2	Projeto Orientado a Objeto – POO	22
4.3	Diagrama de Componentes	24

4.4	Diagrama de Implantação	26
5	Lista das Características/<i>Features</i>	27
5.1	Lista de características <<features>>	27
A	Título do Apêndice	30
A.1	Sub-Título do Apêndice	30
B	Título do Apêndice	31
B.1	Sub-Título do Apêndice	31

Lista de Figuras

1.1	Metodologia utilizada no desenvolvimento do sistema	6
1.2	Diagrama de caso de uso – Caso de uso geral	8
1.3	Diagrama de caso de uso específico	9
2.1	Diagrama de Pacotes	12
3.1	Diagrama de classes	13
3.2	Diagrama de seqüência geral	16
3.3	Diagrama de seqüência específico	16
3.4	Diagrama de comunicação	17
3.5	Diagrama de máquina de estado	18
3.6	Diagrama de atividades	19
4.1	Diagrama de componentes	25
4.2	Diagrama de implantação	26

Lista de Tabelas

1.1	Caso de uso 1	8
-----	-------------------------	---

Capítulo 1

Concepção

Apresenta-se neste capítulo do projeto de engenharia a concepção, a especificação do sistema a ser modelado e desenvolvido.

1.1 Metodologia

A Figura 1.1 apresenta a metodologia a ser utilizada no desenvolvimento do sistema.

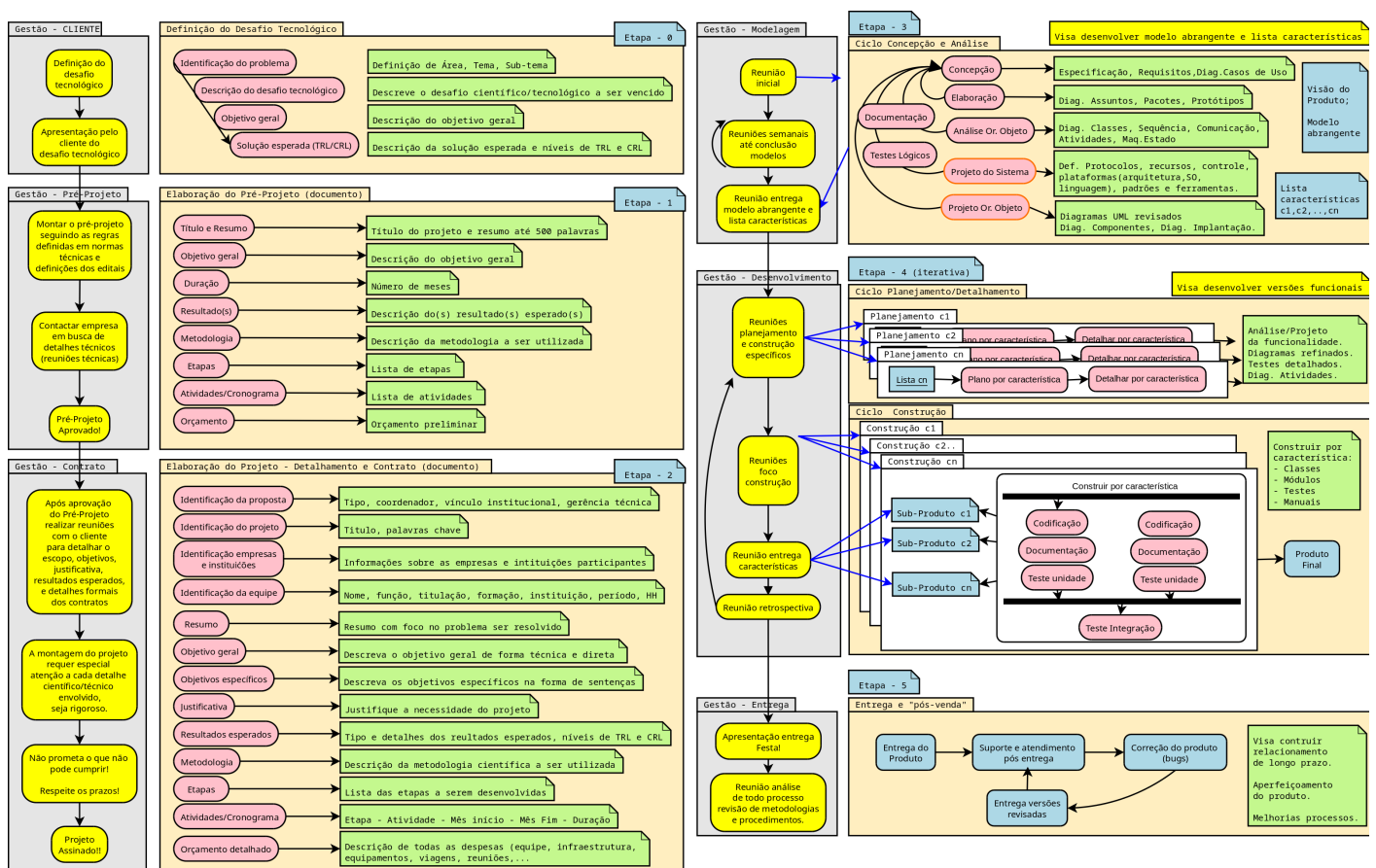


Figura 1.1: Metodologia utilizada no desenvolvimento do sistema

1.2 Nome do Sistema/Produto

Nome	PoroElasticCalculator
Componentes principais	Software inteiramente digital
Missão	Calcular propriedades elásticas de amostras de rocha

1.3 Especificação

Plotar uma série de gráficos que ajude no estudo das propriedades elásticas de rochas

1.4 Requisitos

Apresenta-se nesta seção os requisitos funcionais e não funcionais.

1.4.1 Requisitos funcionais

Apresenta-se a seguir os requisitos funcionais.

RF-01	O sistema deve conter uma base de dados dos módulos de cisalhamento e compressibilidade de vários minerais.
RF-02	O usuário deverá ter liberdade para colocar manualmente os valores de sua amostra ou carregar um arquivo de saída que já contenha estes dados.
RF-03	Deve permitir o carregamento de arquivos criados pelo software.
RF-04	Deve permitir a escolha das equações de Hashin-Strikeman ou Voig-Reus-Hill.
RF-05	O O usuário deve ter tal liberdade para escolher as porcentagens mineralógicas da sua amostra.
RF-06	O usuário poderá plotar seus resultados em um gráfico. O gráfico poderá ser salvo como imagem ou ter seus dados exportados como texto.

1.4.2 Requisitos não funcionais

RNF-01	Os cálculos devem ser feitos utilizando-se as equações de Hashin-Strikeman ou Voig-Reus-Hill.
RNF-02	O programa deverá ser multi-plataforma, podendo ser executado em <i>Windows</i> , <i>GNU/Linux</i> ou <i>MacOS</i> .

1.5 Casos de Uso

A Tabela 1.1 mostra a descrição de um caso de uso.

Tabela 1.1: Caso de uso 1

Nome do caso de uso:	Geral
Resumo/descrição:	Etapas a serem cumpridas pelo usuário
Etapas:	Criar a amostra, carregar os arquivos, gerar os crossplot, analisar os resultados.
Cenários alternativos:	O usuário escolhe os valores manualmente, podendo adicionar novos minerais

1.5.1 Diagrama de caso de uso geral

O diagrama de caso de uso geral da Figura 1.2 mostra o usuário acessando os sistemas de ajuda do software, calculando as propriedades elásticas e analisando os resultados.

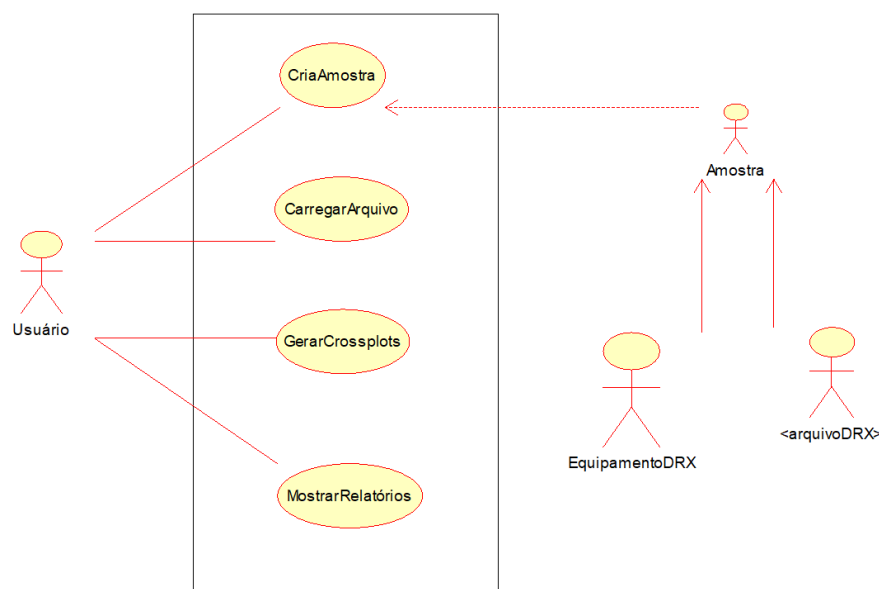


Figura 1.2: Diagrama de caso de uso – Caso de uso geral

1.5.2 Diagrama de caso de uso específico

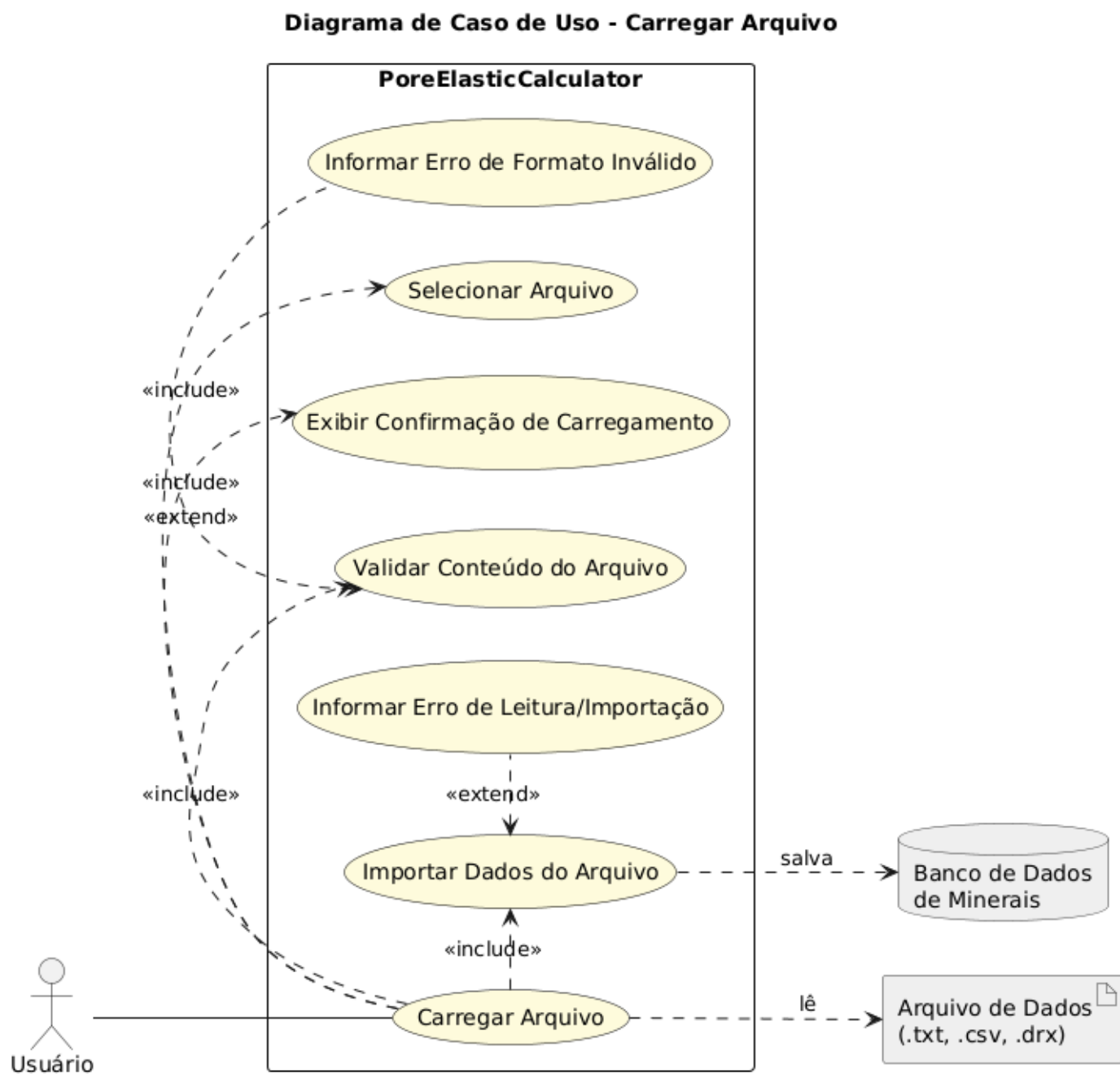


Figura 1.3: Diagrama de caso de uso específico

Capítulo 2

Elaboração

2.1 Análise de domínio

Após estudo dos requisitos/especificações do sistema, algumas entrevistas, estudos na biblioteca e disciplinas do curso foi possível identificar nosso domínio de trabalho:

- Área de aplicação: Engenharia de Petróleo, Petrofísica, Geomecânica
- Problema central: Determinar propriedades elásticas (módulo de cisalhamento e compressibilidade) de rochas com composições mineralógicas variadas, algo comum em formações sedimentares.

2.2 Formulação teórica

A determinação das propriedades elásticas efetivas de rochas com múltiplos constituintes minerais exige o uso de modelos de homogeneização que aproximem o comportamento macroscópico do meio composto. Neste trabalho, são utilizados os modelos de Voigt-Reuss-Hill (VRH) e os limites de Hashin-Shtrikman (HS) para a estimativa do módulo de compressibilidade K e do módulo de cisalhamento G da rocha.

2.2.1 Modelo de Voigt-Reuss-Hill (VRH)

O modelo de Voigt assume deformação uniforme entre os constituintes da mistura, resultando em uma média aritmética ponderada das propriedades dos minerais:

$$K_V = \sum_{i=1}^n f_i K_i \quad (2.1)$$

$$G_V = \sum_{i=1}^n f_i G_i \quad (2.2)$$

Já o modelo de Reuss considera tensão uniforme entre os constituintes, levando a uma média harmônica ponderada:

$$\frac{1}{K_R} = \sum_{i=1}^n \frac{f_i}{K_i} \quad (2.3)$$

$$\frac{1}{G_R} = \sum_{i=1}^n \frac{f_i}{G_i} \quad (2.4)$$

A média de Voigt-Reuss-Hill (VRH) é obtida como a média aritmética simples entre os valores de Voigt e Reuss:

$$K_{VRH} = \frac{K_V + K_R}{2} \quad (2.5)$$

$$G_{VRH} = \frac{G_V + G_R}{2} \quad (2.6)$$

2.2.2 Limites de Hashin-Shtrikman (HS)

Os limites de Hashin-Shtrikman oferecem uma estimativa teórica rigorosa dos valores mínimo e máximo possíveis para as propriedades elásticas de materiais isotrópicos compostos por duas ou mais fases. Supondo K_1 e G_1 como os módulos do mineral com maior rigidez e K_2 e G_2 como os módulos do mineral mais fraco, com frações volumétricas f_1 e f_2 , os limites são dados por:

$$K_{HS}^+ = K_2 + \frac{f_1}{1K_1 - K_2 + f_2K_2 + 43G_2} \quad (2.7)$$

$$G_{HS}^+ = G_2 + \frac{f_1}{1G_1 - G_2 + f_2G_2 + \zeta} \quad (2.8)$$

$$K_{HS}^- = K_1 + \frac{f_2}{1K_2 - K_1 + f_1K_1 + 43G_1} \quad (2.9)$$

$$G_{HS}^- = G_1 + \frac{f_2}{1G_2 - G_1 + f_1G_1 + \zeta'} \quad (2.10)$$

Onde:

$$\zeta = \frac{G_2(9K_2 + 8G_2)}{6(K_2 + 2G_2)} \quad ; \quad \zeta' = \frac{G_1(9K_1 + 8G_1)}{6(K_1 + 2G_1)} \quad (2.11)$$

Esses limites fornecem uma faixa possível para as propriedades efetivas, sendo úteis para validar os resultados obtidos com modelos mais simplificados como VRH.

2.3 Identificação de pacotes – assuntos

Este projeto fundamenta-se em um conjunto de pacotes teóricos e computacionais voltados para o estudo das propriedades elásticas de rochas com múltiplas composições mineralógicas. A seguir são descritos os principais pacotes envolvidos, com sua respectiva função no desenvolvimento do modelo.

- Rock Physics: Conjunto de teorias que conectam propriedades elásticas da rocha (como módulo de cisalhamento e compressibilidade) com suas características petrofísicas, como composição mi-

neralógica, porosidade e saturação. Este pacote fornece a base conceitual para a formulação dos modelos de mistura empregados neste trabalho.

- Mineral Physics: Ramo que estuda as propriedades físicas dos minerais puros, como o módulo de compressibilidade (K) e o módulo de cisalhamento (G). Esses valores são fundamentais como dados de entrada para os modelos de homogeneização. Os dados típicos são obtidos da literatura especializada.
- Método de homogeneização que fornece uma média das propriedades elásticas baseada em hipóteses de deformação (Voigt) e tensão (Reuss) uniformes. A média de Voigt-Reuss-Hill representa uma estimativa intermediária entre os extremos teóricos.
- Gnuplot, ferramenta de visualização utilizada para representar graficamente os resultados dos modelos, como comparação entre VRH e HS, análises de sensibilidade e tendências com variação mineralógica.
- Teoria dos meios efetivos: Pacote teórico que abrange todos os modelos de homogeneização para materiais compostos, como VRH, HS, DEM (Differential Effective Medium) e outros. Justifica conceitualmente o uso dos modelos implementados neste trabalho.

2.4 Diagrama de pacotes – assuntos

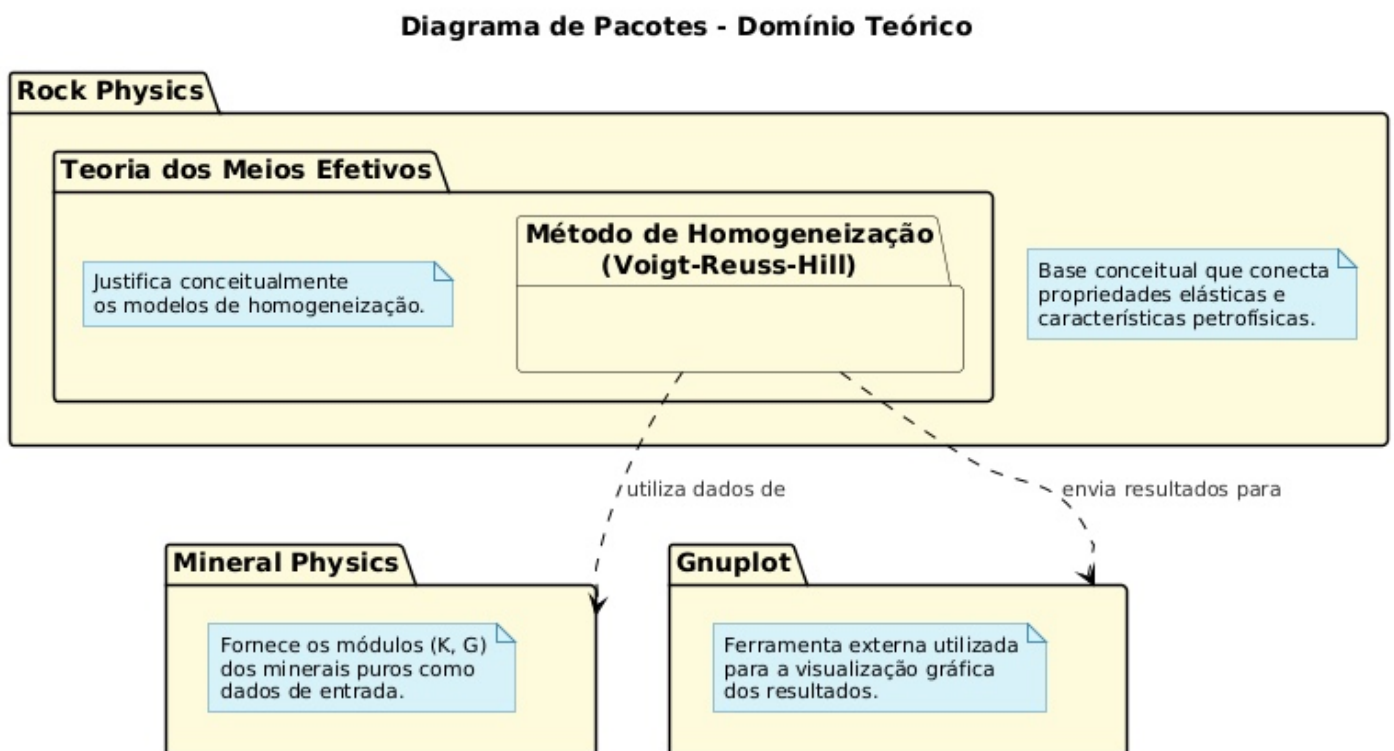


Figura 2.1: Diagrama de Pacotes

Capítulo 3

AOO – Análise Orientada a Objeto

3.1 Diagramas de classes

O diagrama de classes é apresentado na Figura 3.1.

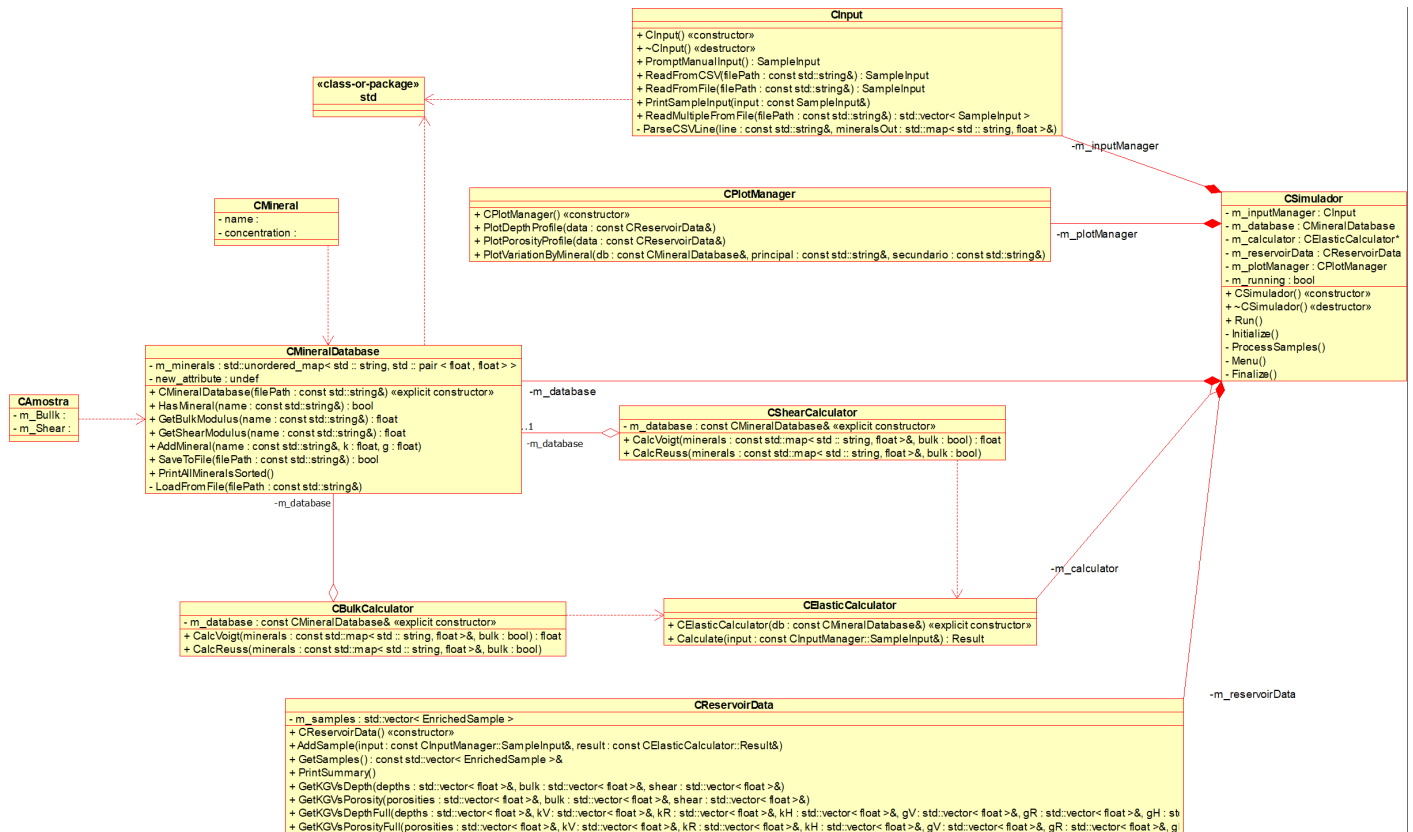


Figura 3.1: Diagrama de classes

3.1.1 Dicionário de classes

- CSimulator

Responsabilidade: Classe principal que orquestra a execução do programa. É responsável por inicializar todos os componentes, gerenciar o menu de interação com o usuário e controlar o fluxo de dados entre os módulos de entrada, cálculo e visualização.

Atributos: -minput: CInput: Instância do gerenciador de entrada de dados. -mdatabase: CMineralDatabase: Instância do banco de dados de minerais. -mcalculator: CElasticCalculator: Instância do módulo de cálculo de propriedades elásticas. -mreservoirData: CReservoirData: Instância do repositório de dados das amostras processadas. -mplotManager: CPlotManager: Instância do gerenciador de visualização gráfica.

Métodos: +CSimulator() / + CSimulator(): Construtor e destrutor da classe. +Run(): Inicia e gerencia o laço de execução principal da aplicação. -Initialize(): Prepara e aloca os recursos para os módulos do sistema. -Menu(): Exibe as opções disponíveis e captura a escolha do usuário. -ProcessSamples(): Dispara o fluxo de processamento para uma nova amostra. -Finalize(): Libera os recursos alocados antes de encerrar a aplicação.

- CInput

Responsabilidade: Gerencia a aquisição de dados das amostras, seja por meio de entrada manual do usuário no console ou pela leitura e interpretação de arquivos de dados formatados.

Métodos: +CInput() / + CInput(): Construtor e destrutor da classe. +PromptManualInput(): Solicita interativamente os dados da amostra ao usuário. +ReadFromCSV(): Lê dados de um arquivo no formato CSV (não utilizado na versão final, conforme código-fonte). +ReadFromFile(): Lê os dados de uma única amostra de um arquivo de texto. +PrintSampleInput(): Exibe os dados de entrada de uma amostra no console para verificação. +ReadMultipleFromFile(): Lê uma ou mais amostras de um arquivo de texto. -ParseCSVLine(): Interpreta uma única linha de um arquivo de dados para extrair a composição mineralógica.

- CMineralDatabase

Responsabilidade: Encapsula o acesso à base de dados de minerais. É responsável por carregar, salvar e fornecer as propriedades elásticas (módulo de compressibilidade e de cisalhamento) de cada mineral.

Atributos: -mminerals: Estrutura de dados que armazena as propriedades dos minerais carregados em memória. -mdatabaseFilePath: Caminho para o arquivo físico da base de dados. Métodos: +CMineralDatabase(filePath): Construtor que inicializa o objeto e carrega os dados do arquivo especificado. +HasMineral(name): Verifica a existência de um mineral na base de dados. +GetBulkModulus(name): Retorna o módulo de compressibilidade (K) de um mineral. +GetShearModulus(name): Retorna o módulo de cisalhamento (G) de um mineral. +AddMineral(name, k, g): Adiciona um novo mineral à base de dados em memória. +SaveToFile(filePath): Salva o estado atual da base de dados em um arquivo. +PrintAllMineralsSorted(): Exibe todos os minerais da base de dados em ordem alfabética. -LoadFromFile(filePath): Lógica interna para ler e interpretar o arquivo da base de dados.

- CElasticCalculator

Responsabilidade: Constitui o núcleo de cálculo do sistema. Implementa os modelos de homogeneização de Voigt e Reuss para estimar os módulos elásticos efetivos da matriz rochosa.

Atributos: -mdatabase: CMineralDatabase: Referência para a base de dados de minerais, utili-

zada para consultar as propriedades durante o cálculo.

Métodos: +CElasticCalculator(db): Construtor que recebe uma referência ao banco de dados de minerais. +Calculate(input): Orquestra o cálculo para uma amostra, invocando os métodos de Voigt e Reuss e retornando os resultados. -CalcVoigt(minerals, bulk): Implementa a média de Voigt para o módulo de compressibilidade ou de cisalhamento. -CalcReuss(minerals, bulk): Implementa a média de Reuss para o módulo de compressibilidade ou de cisalhamento.

- CReservoirData

Responsabilidade: Atua como um repositório para as amostras processadas. Armazena os dados de entrada junto com os resultados calculados para uso posterior em relatórios e gráficos.

Atributos: -msamples: Vetor que armazena o conjunto de todas as amostras processadas durante a sessão.

Métodos: +CReservoirData(): Construtor da classe. +AddSample(input, result): Adiciona uma nova amostra e seus resultados calculados ao repositório. +GetSamples(): Retorna uma referência para o vetor completo de amostras. +PrintSummary(): Exibe no console um resumo com os principais resultados de todas as amostras. +GetKGVvsDepth(): Extrai e organiza os dados para plotagem de propriedades vs. profundidade. +GetKGVvsPorosity(): Extrai e organiza os dados para plotagem de propriedades vs. porosidade. +GetKGVvsDepthFull(): Retorna todos os limites (Voigt, Reuss, Hill) para os perfis de profundidade. +GetKGVvsPorosityFull(): Retorna todos os limites (Voigt, Reuss, Hill) para os perfis de porosidade.

- CPlotManager

Responsabilidade: Módulo dedicado à visualização de dados. Utiliza os resultados consolidados em CReservoirData para gerar e exibir gráficos, facilitando a análise e interpretação dos resultados.

Métodos: +CPlotManager(): Construtor da classe. +PlotDepthProfile(data): Gera o gráfico dos módulos elásticos em função da profundidade para as amostras fornecidas. +PlotPorosityProfile(data): Gera o gráfico dos módulos elásticos em função da porosidade. +PlotVariationByMineral(db, ...): Cria gráficos de sensibilidade à variação percentual entre dois minerais.

3.2 Diagrama de seqüência – eventos e mensagens

3.2.1 Diagrama de seqüência geral

Veja o diagrama de seqüência na Figura 3.2.

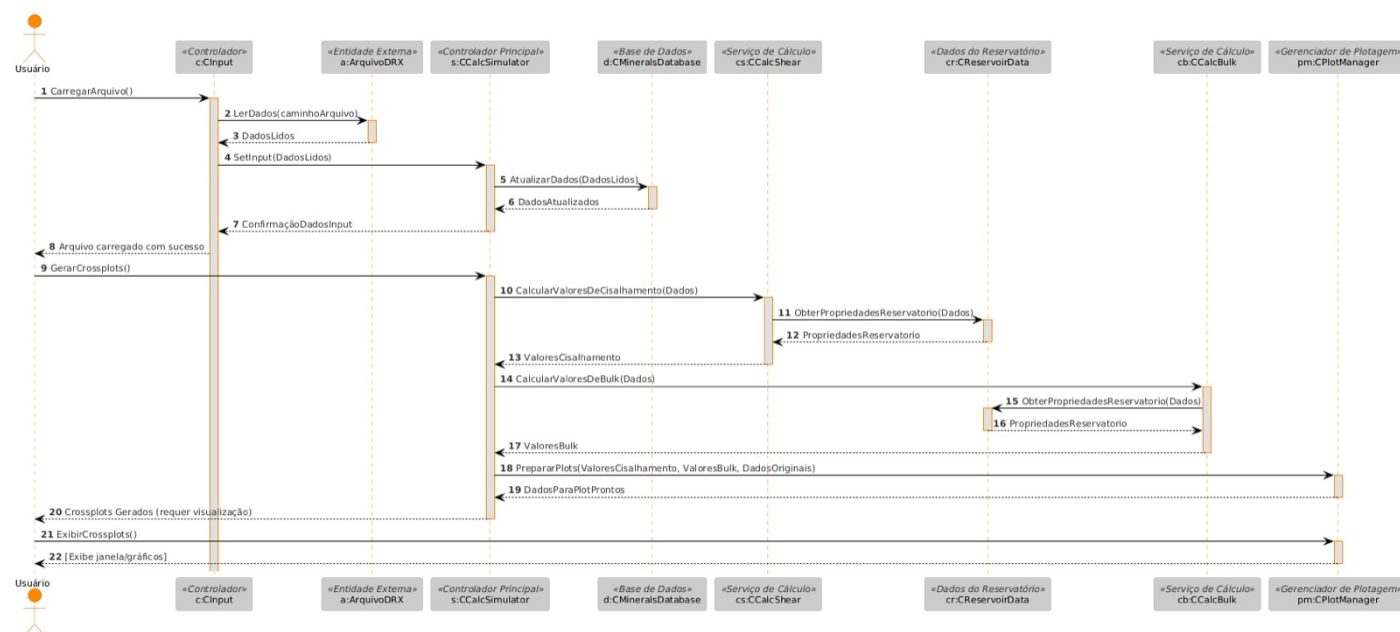


Figura 3.2: Diagrama de sequência geral

3.2.2 Diagrama de sequência específico

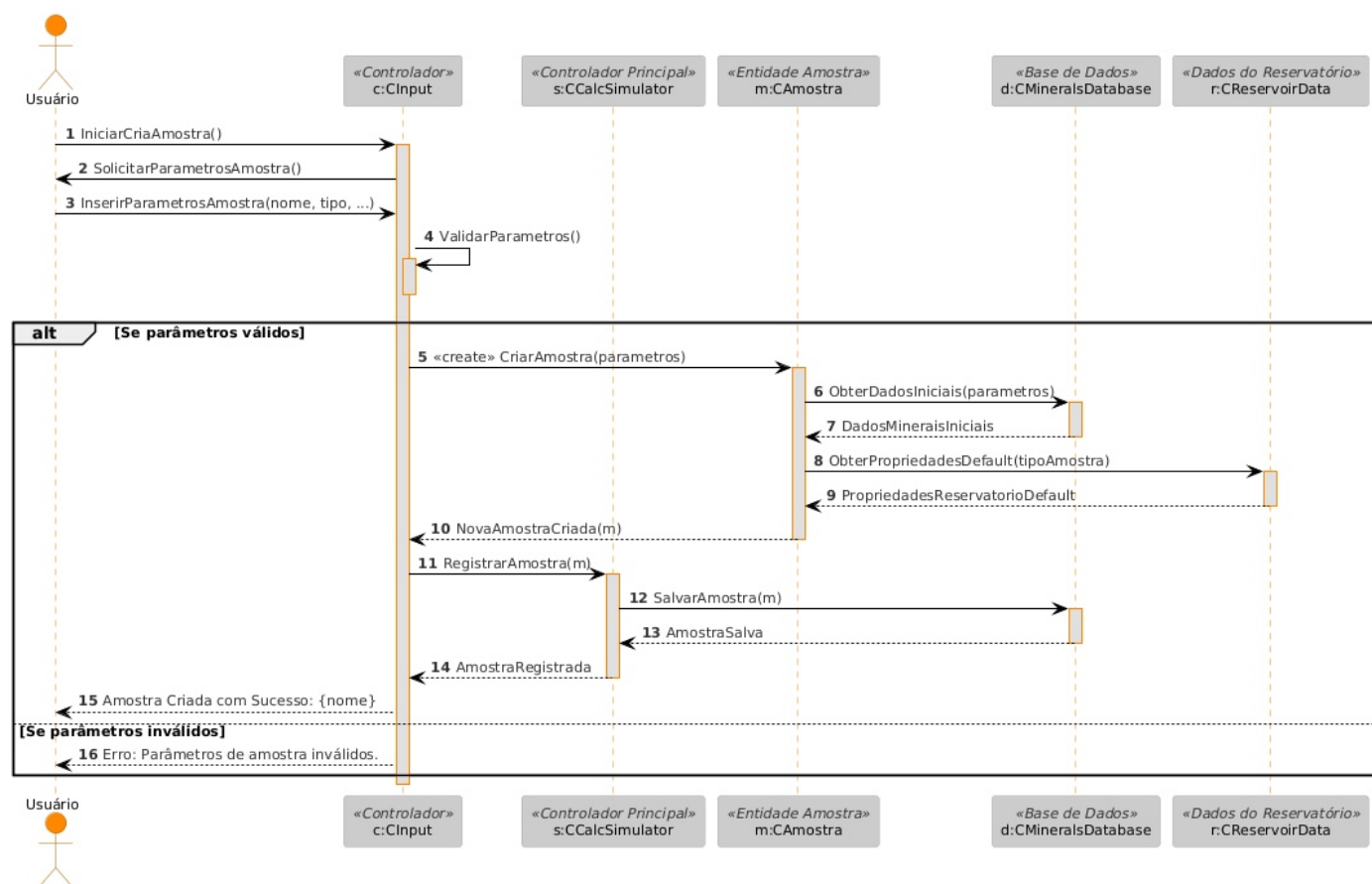


Figura 3.3: Diagrama de sequência específico

3.3 Diagrama de comunicação – colaboração

No diagrama de comunicação o foco é a interação e a troca de mensagens e dados entre os objetos.

Veja na Figura 3.3 o diagrama de comunicação mostrando a sequência de interação, onde a aplicação solicita leitura do arquivo da amostra para dar prosseguimento com o cálculo das propriedades, onde busca os dados do banco de dados. A aplicação armazena os dados e gera os resultados, e por fim solicita a geração dos gráficos de perfil.

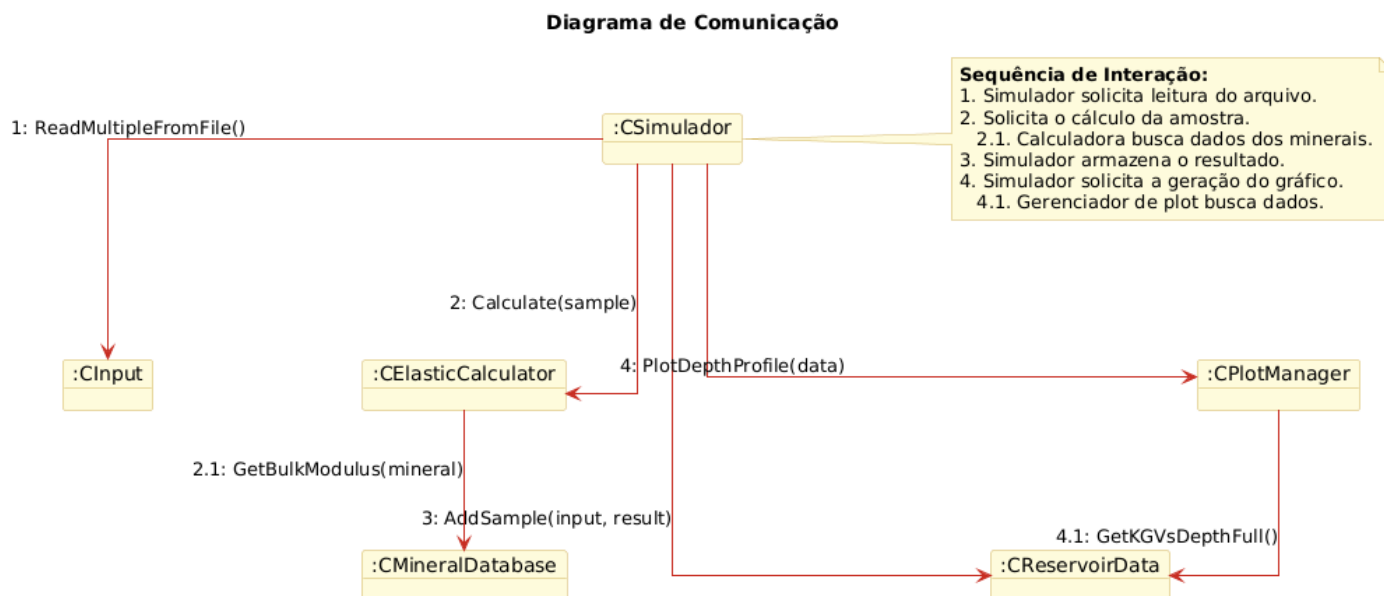


Figura 3.4: Diagrama de comunicação

3.4 Diagrama de estado

Veja na Figura 3.5 o diagrama de máquina de estado para o objeto CSimulador

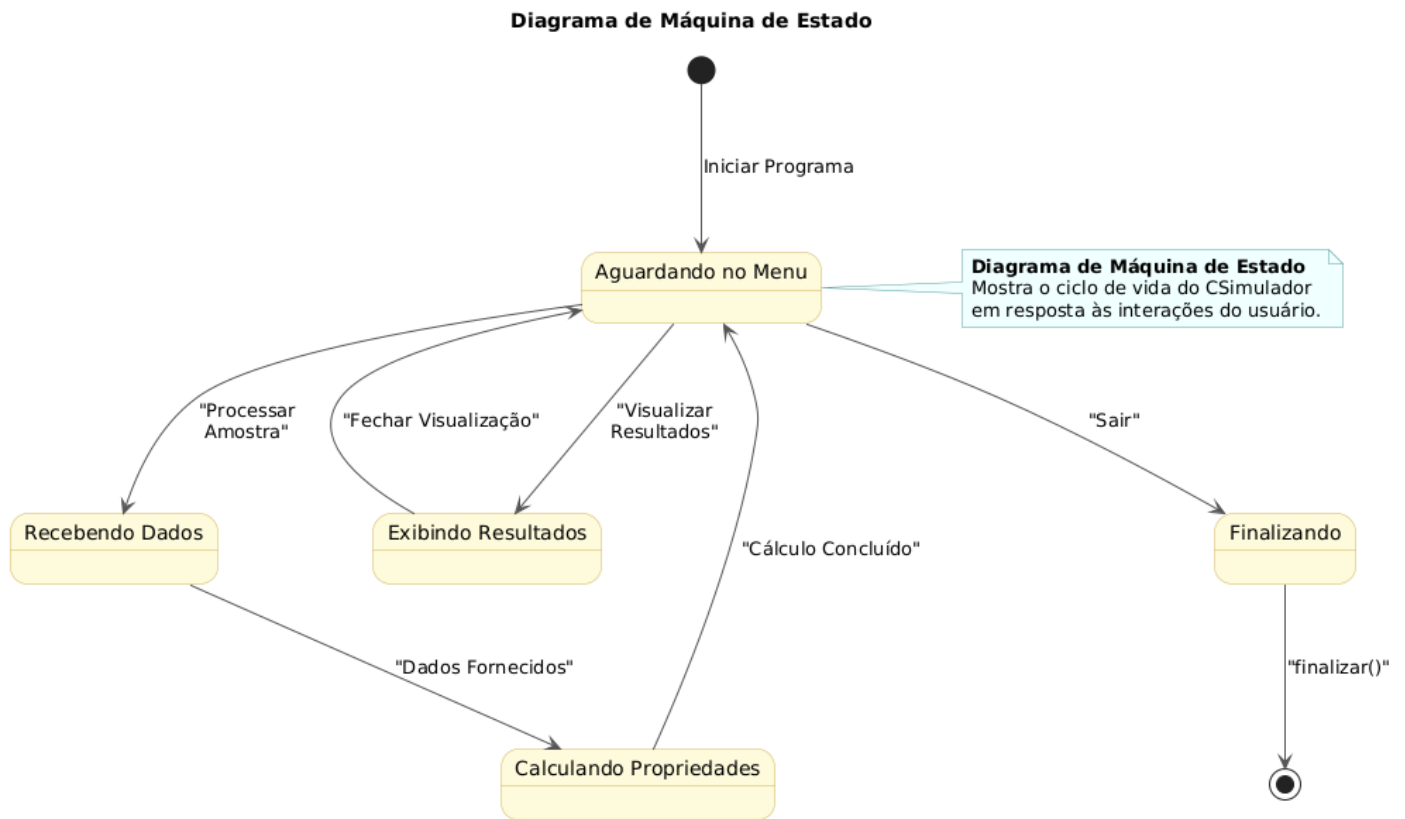


Figura 3.5: Diagrama de máquina de estado

3.5 Diagrama de atividades

Veja na Figura 3.6 o diagrama de atividades.

O processo se inicia no Nó Inicial, levando diretamente à primeira Atividade, que é "Apresentar Menu Principal". Neste ponto, uma decisão importante é oferecida ao usuário: se a opção for "Sair", o fluxo é desviado diretamente para a finalização do programa; caso contrário, ao escolher "Processar Amostra", o sistema avança para uma segunda decisão que define o método de entrada dos dados, seja através da atividade "Carregar de Arquivo" ou da criação manual. Uma vez que os dados da amostra são fornecidos por um desses caminhos, o fluxo converge e prossegue de forma sequencial para a fase de processamento, onde a atividade central de "Executar cálculos de propriedades poroelásticas" é realizada.

Com o término dos cálculos, indicado pelo estado "Resultados calculados e disponíveis", uma nova decisão permite ao usuário escolher entre "Gerar Gráficos" ou "Mostrar Relatórios" para a análise final. Concluída esta etapa, o diagrama chega ao seu ciclo de execução principal, onde a última decisão do fluxo pergunta se o usuário "Deseja realizar outra operação?". Uma resposta afirmativa reinicia todo o processo, retornando o controle ao menu principal para um novo ciclo de trabalho. Caso a resposta seja negativa, o fluxo segue para a atividade "Finalizar Software" e culmina no Nó Final, que representa o encerramento completo da aplicação.

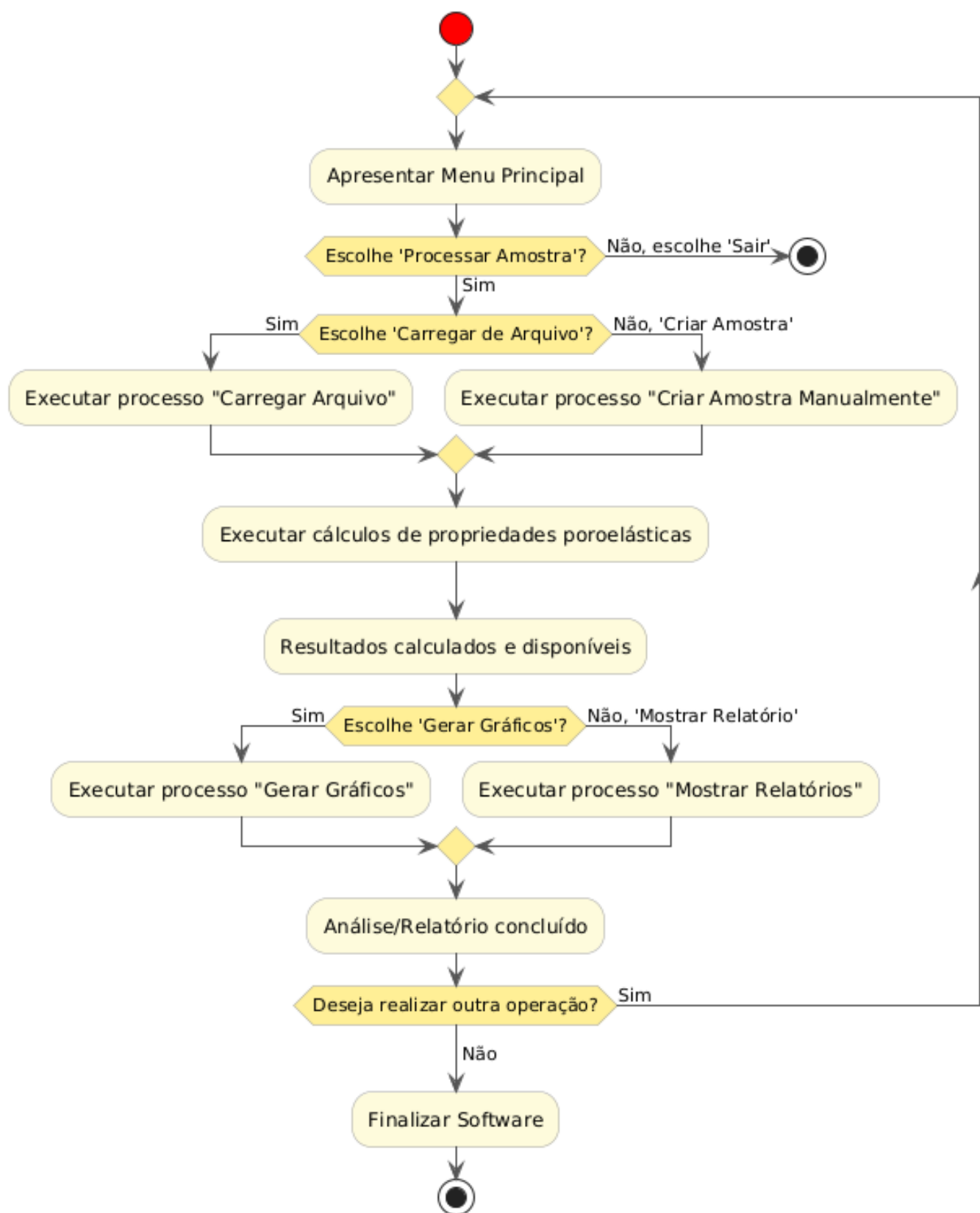
Diagrama de Atividade - Fluxo do Software

Figura 3.6: Diagrama de atividades

Capítulo 4

Projeto

Neste capítulo do projeto de engenharia veremos questões associadas ao projeto do sistema, incluindo protocolos, recursos, plataformas suportadas, implicações nos diagramas feitos anteriormente, diagramas de componentes e implantação. Na segunda parte revisamos os diagramas levando em conta as decisões do projeto do sistema.

4.1 Projeto do Sistema

Depois da análise orientada a objeto desenvolve-se o projeto do sistema, qual envolve etapas como a definição dos protocolos, da interface API, o uso de recursos, a subdivisão do sistema em subsistemas, a alocação dos subsistemas ao hardware e a seleção das estruturas de controle, a seleção das plataformas do sistema, das bibliotecas externas, dos padrões de projeto, além da tomada de decisões conceituais e políticas que formam a infraestrutura do projeto.

Deve-se definir padrões de documentação, padrões para o nome das classes, padrões de retorno e de parâmetros em métodos, características da interface do usuário e características de desempenho.

O projeto do sistema é a estratégia de alto nível para resolver o problema e elaborar uma solução.

1. Protocolos

- O sistema é concebido para ser uma aplicação desktop autônoma, com capacidade de importar dados e exportar resultados. A comunicação com dispositivos externos (como sensores laboratoriais, espectrômetros, etc.) não será necessária nesta primeira versão. No entanto, o software deverá:
 - Ler arquivos de entrada com composições mineralógicas em formatos abertos, preferencialmente .csv e .xlsx
 - Exportar os resultados dos cálculos em formatos igualmente abertos em .txt (relatórios simples); .csv (planilhas com resultados); .png (gráficos e visualizações)
- O sistema será implementado seguindo o paradigma de programação orientada a objetos, sendo os principais componentes do sistema representados como objetos com responsabilidades bem definidas.

A comunicação entre objetos seguirá os princípios do acoplamento fraco e alta coesão, utilizando:

- Chamadas diretas de métodos
- Passagem de parâmetros por instância (composição e agregação)
- Padrão de projeto MVC (Model-View-Controller), separando: Modelos físicos e matemáticos (Model); Interface com o usuário (View); Lógica de controle e fluxo de dados (Controller)
- O sistema utilizará os seguintes formatos abertos para arquivos de entrada:
 - .csv: para dados de composição mineralógica, módulos dos minerais, nomes de fases, etc.
 - .json: (opcional) para configurações de execução (modelo a usar, limites, parâmetros fixos)
- O sistema utilizará os seguintes formatos abertos para arquivos de saída:
 - .csv: resultados tabulares
 - .txt: logs e relatórios legíveis
 - .png / .pdf: gráficos comparativos
- Recursos
 - O sistema utiliza um banco de dados embutido contendo as propriedades elásticas conhecidas dos minerais (módulo de compressibilidade K, módulo de cisalhamento G, profundidade da amostra, porosidade). Este banco de dados permite a automação da busca de propriedades físicas a partir da identificação do nome do mineral, eliminando a necessidade de inserção manual dos parâmetros. Exemplo de estrutura da tabela minerais:

nome	k	g
albita	55	29.5
anidrita	55	30
anortita	84	40
aragonita	47	38.5
⋮	⋮	⋮

- Plataformas
 - * Padrão de Nomes
 - Classes: CamelCase, ex: CInput
 - Métodos: snakecase, ex: ReadFromCSV
 - * Padrão de retorno e parâmetros, que devem retornar:
 - Objetos contendo os resultados (dicionários ou instâncias de classes)
 - Em caso de erro, exceções customizadas devem ser lançadas

4.2 Projeto Orientado a Objeto – POO

O projeto orientado a objeto é a etapa posterior ao projeto do sistema. Baseia-se na análise, mas considera as decisões do projeto do sistema. Acrescenta a análise desenvolvida e as características da plataforma escolhida (hardware, sistema operacional e linguagem de softwareção). Passa pelo maior detalhamento do funcionamento do software, acrescentando atributos e métodos que envolvem a solução de problemas específicos não identificados durante a análise.

Envolve a otimização da estrutura de dados e dos algoritmos, a minimização do tempo de execução, de memória e de custos. Existe um desvio de ênfase para os conceitos da plataforma selecionada.

Exemplo: na análise você define que existe um método para salvar um arquivo em disco, define um atributo nomeDoArquivo, mas não se preocupa com detalhes específicos da linguagem. Já no projeto, você inclui as bibliotecas necessárias para acesso ao disco, cria um objeto específico para acessar o disco, podendo, portanto, acrescentar novas classes àquelas desenvolvidas na análise.

Efeitos do projeto no modelo estrutural

- Adicionar nos diagramas de pacotes as bibliotecas e subsistemas selecionados no projeto do sistema (exemplo: a biblioteca gráfica selecionada).
 - * Neste projeto blablabla
- Novas classes e associações oriundas das bibliotecas selecionadas e da linguagem escolhida devem ser acrescentadas ao modelo.
 - * Neste projeto blablabla
- Estabelecer as dependências e restrições associadas à plataforma escolhida.
 - * Neste projeto blablabla

Efeitos do projeto no modelo dinâmico

- Revisar os diagramas de seqüência e de comunicação considerando a plataforma escolhida.
 - * Neste projeto blablabla
- Verificar a necessidade de se revisar, ampliar e adicionar novos diagramas de máquinas de estado e de atividades.
 - * Neste projeto blablabla

Efeitos do projeto nos atributos

- Atributos novos podem ser adicionados a uma classe, como, por exemplo, atributos específicos de uma determinada linguagem de softwareção (acesso a disco, ponteiros, constantes e informações correlacionadas).

- * Neste projeto blablabla

Efeitos do projeto nos métodos

- Em função da plataforma escolhida, verifique as possíveis alterações nos métodos. O projeto do sistema costuma afetar os métodos de acesso aos diversos dispositivos (exemplo: hd, rede).
 - * Neste projeto blablabla
- De maneira geral os métodos devem ser divididos em dois tipos: i) tomada de decisões, métodos políticos ou de controle; devem ser claros, legíveis, flexíveis e usam polimorfismo. ii) realização de processamentos, podem ser otimizados e em alguns casos o polimorfismo deve ser evitado.
 - * Neste projeto blablabla
- Algoritmos complexos podem ser subdivididos. Verifique quais métodos podem ser otimizados. Pense em utilizar algoritmos prontos como os da STL (algoritmos genéricos).
 - * Neste projeto blablabla
- Responda a pergunta: os métodos da classes estão dando resposta às responsabilidades da classe?
 - * Neste projeto blablabla
- Revise os diagramas de classes, de sequência e de máquina de estado.
 - * Neste projeto blablabla

Efeitos do projeto nas heranças

- Reorganização das classes e dos métodos (criar métodos genéricos com parâmetros que nem sempre são necessários e englobam métodos existentes).
 - * Neste projeto blablabla
- Abstração do comportamento comum (duas classes podem ter uma superclasse em comum).
 - * Neste projeto blablabla
- Utilização de delegação para compartilhar a implementação (quando você cria uma herança irreal para reaproveitar código). Usar com cuidado.
 - * Neste projeto blablabla
- Revise as heranças no diagrama de classes.
 - * Neste projeto blablabla

Efeitos do projeto nas associações

- Deve-se definir na fase de projeto como as associações serão implementadas, se obedecerão um determinado padrão ou não.

- * Neste projeto blablabla
- Se existe uma relação de "muitos", pode-se implementar a associação com a utilização de um dicionário, que é um mapa das associações entre objetos. Assim, o objeto A acessa o dicionário fornecendo uma chave (um nome para o objeto que deseja acessar) e o dicionário retorna um valor (um ponteiro) para o objeto correto.
- * Neste projeto blablabla
- Evite percorrer várias associações para acessar dados de classes distantes. Pense em adicionar associações diretas.
- * Neste projeto blablabla

Efeitos do projeto nas otimizações

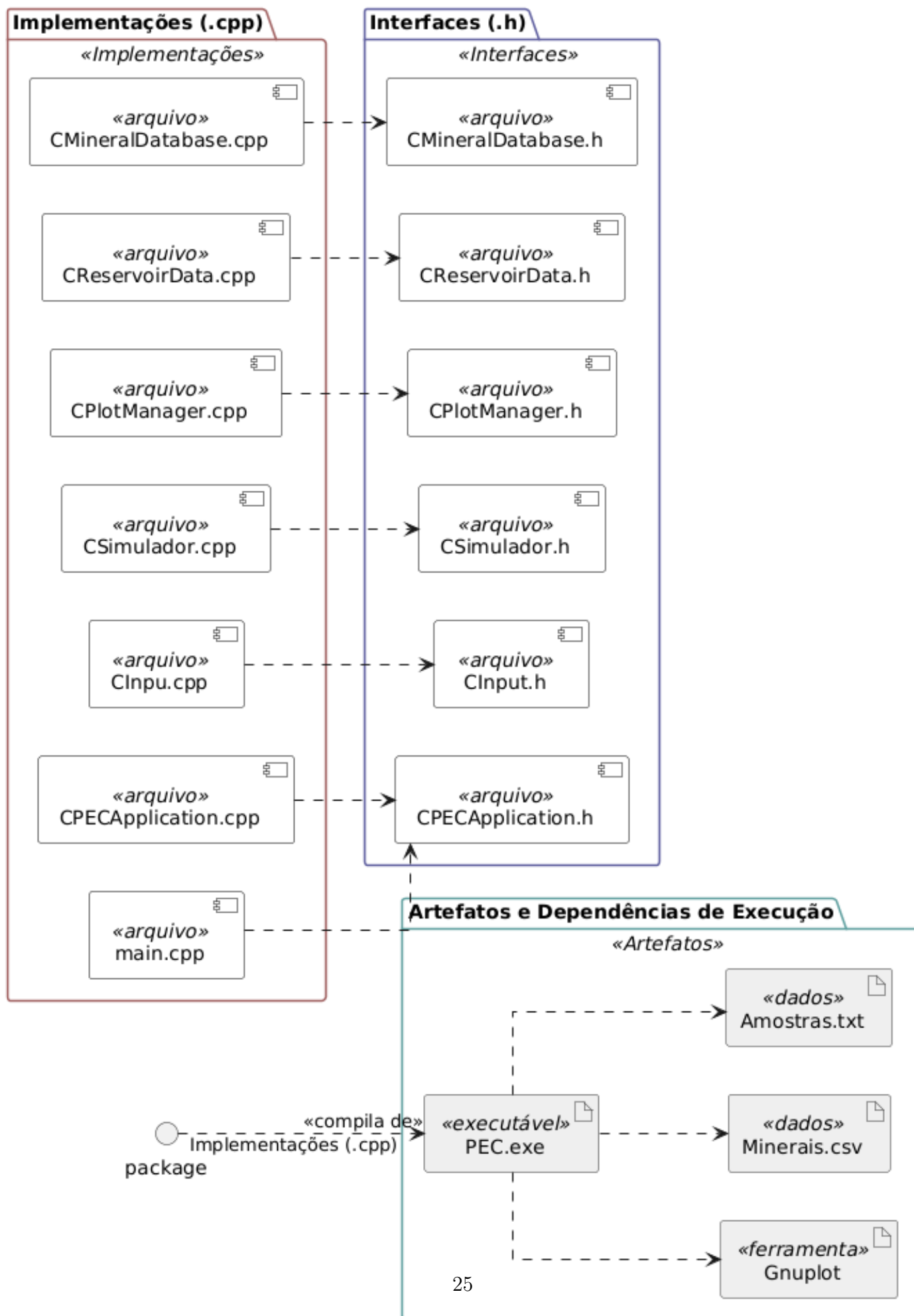
- Faça uma análise de aspectos relativos à otimização do sistema. Lembrando que a otimização deve ser desenvolvida por analistas/desenvolvedores experientes.
- * Neste projeto blablabla
- Identifique pontos a serem otimizados em que podem ser utilizados processos concorrentes.
- * Neste projeto blablabla
- Pense em incluir bibliotecas otimizadas.
- Se o acesso a determinados objetos (atributos/métodos) requer um caminho longo (exemplo: A->B->C->D.atributo), pense em incluir associações extras (exemplo: A-D.atributo).
- * Neste projeto blablabla
- Atributos auxiliares podem ser incluídos.
- * Neste projeto blablabla
- A ordem de execução pode ser alterada.
- * Neste projeto blablabla
- Revise as associações nos diagramas de classes.
- * Neste projeto blablabla

Depois de revisados os diagramas da análise você pode montar dois diagramas relacionados à infraestrutura do sistema. As dependências dos arquivos e bibliotecas podem ser descritos pelo diagrama de componentes, e as relações e dependências entre o sistema e o hardware podem ser ilustradas com o diagrama de implantação.

4.3 Diagrama de Componentes

Segue o diagrama de componentes:

Diagrama de Componentes



4.4 Diagrama de Implantação

O diagrama de implantação é um diagrama de alto nível que inclui relações entre o sistema e o hardware e que se preocupa com os aspectos da arquitetura computacional escolhida. Seu enfoque é o hardware, a configuração dos nós em tempo de execução.

O diagrama de implantação deve incluir os elementos necessários para que o sistema seja colocado em funcionamento: computador, periféricos, processadores, dispositivos, nós, relacionamentos de dependência, associação, componentes, subsistemas, restrições e notas.

Veja na Figura 4.2 um exemplo de diagrama de implantação de um cluster. Observe a presença de um servidor conectado a um switch. Os nós do cluster (ou clientes) também estão conectados ao switch. Os resultados das simulações são armazenados em um servidor de arquivos (*storage*).

Pode-se utilizar uma anotação de localização para identificar onde determinado componente está residente, por exemplo {localização: sala 3}.

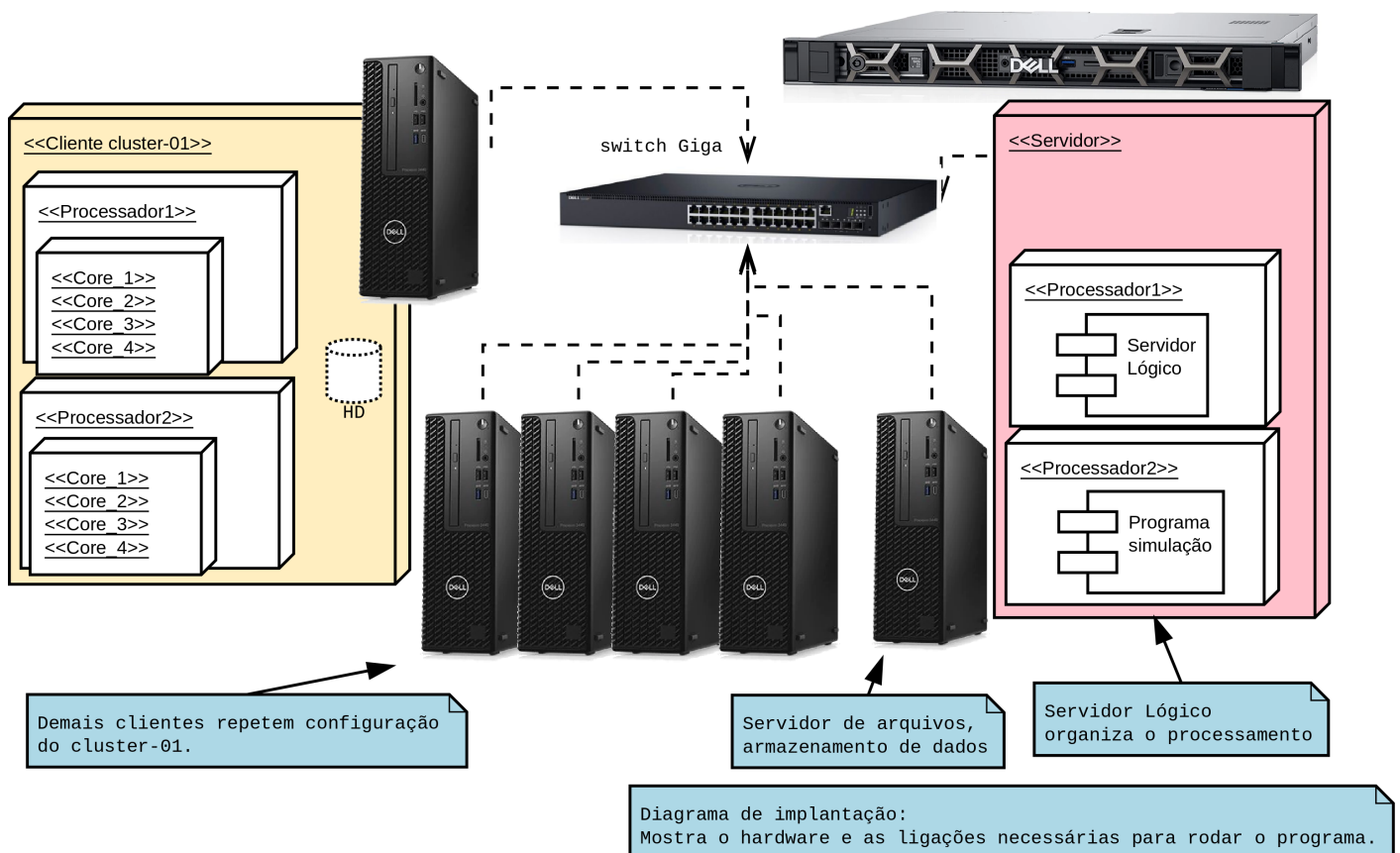


Figura 4.2: Diagrama de implantação

Nota:

Não perca de vista a visão do todo; do projeto de engenharia como um todo. Cada capítulo, cada seção, cada parágrafo deve se encaixar. Este é um diferencial fundamental do engenheiro em relação ao técnico, a capacidade de desenvolver projetos, de ver o todo e suas diferentes partes, de modelar processos/sistemas/produtos de engenharia.

Capítulo 5

Lista das Características/*Features*

Neste capítulo lista-se as características do sistema a ser desenvolvido (seção 5.1).

5.1 Lista de características <<features>>

No final do ciclo de concepção e análise chegamos a uma lista de características <<*features*>> que teremos de implementar.

Após a análises desenvolvidas e considerando o requisito de que este material deve ter um formato didático, chegamos a seguinte lista:

- v0.1
 - * Ex: O sistema deve plotar gráficos de pressão x temperatura
 - * Ex: O sistema deve permitir abrir arquivos num formato especificado.
 - * Ex: nome de classe a ser implementada e testada
- v0.3
 - * Lista de classes a serem implementadas
 - * Testes
- v0.7
 - * Lista de classes a serem implementadas
 - * Testes

Referências Bibliográficas

Apêndice A

Título do Apêndice

Descreve-se neste apêndice ...

- Os anexos ou apêndices contêm material auxiliar. Por exemplo, tabelas, gráficos, resultados de experimentos, algoritmos, códigos e simulações.
- Um apêndice pode incluir assuntos mais gerais (geral demais para estar no núcleo do trabalho) ou mais específicos (detalhado demais para estar no núcleo do trabalho).
- Pode conter um artigo de auxílio fundamental ao trabalho.
- Pode conter artigos publicados.
- [tudo aquilo que for importante para a tese mas não essencial, deve ser colocado em apêndices]
- [como exemplo, revisão de metodologias, técnicas, modelos matemáticos, itens desenvolvidos por terceiros]
- [algoritmos e programas devem ser colocados no apêndice]
- [imagens detalhadas de programas desenvolvidos devem ser colocados no apêndice]

A.1 Sub-Título do Apêndice

.....conteúdo..

Apêndice B

Título do Apêndice

Descreve-se neste apêndice ...

- Os anexos ou apêndices contém material auxiliar. Por exemplo, tabelas, gráficos, resultados de experimentos, algoritmos, códigos e simulações.
- Um apêndice pode incluir assuntos mais gerais (geral demais para estar no núcleo do trabalho) ou mais específicos (detalhado demais para estar no núcleo do trabalho).
- Pode conter um artigo de auxílio fundamental ao trabalho.
- Pode conter artigos publicados.
- [tudo aquilo que for importante para a tese mas não essencial, deve ser colocado em apêndices]
- [como exemplo, revisão de metodologias, técnicas, modelos matemáticos, itens desenvolvidos por terceiros]
- [algoritmos e programas devem ser colocados no apêndice]
- [imagens detalhadas de programas desenvolvidos devem ser colocados no apêndice]

B.1 Sub-Título do Apêndice

.....conteúdo..

Índice Remissivo

Análise orientada a objeto, 13

AOO, 13

Associações, 23

atributos, 22

Casos de uso, 8

colaboração, 17

comunicação, 17

Concepção, 6

Diagrama de colaboração, 17

Diagrama de componentes, 24

Diagrama de estado, 17

Diagrama de execução, 26

Diagrama de implantação, 26

Diagrama de sequência, 15

Efeitos do projeto nas associações, 23

Efeitos do projeto nas heranças, 23

Efeitos do projeto nos métodos, 23

Elaboração, 10

Especificação, 7

especificação, 7

estado, 17

Eventos, 15

Heranças, 23

heranças, 23

Mensagens, 15

modelo, 22

métodos, 23

otimizações, 24

Plataformas, 21

POO, 22

Projeto do sistema, 20

Projeto orientado a objeto, 22

Protocolos, 20

Recursos, 21

Requisitos, 7

Requisitos funcionais, 7

Requisitos não funcionais, 7